Experiment No. 1

Aim:

Data Loading, Storage and File Format
Analyzing Sales data from Multiple File Format. Sales data in multiple file formats (e.g., CSV, Excel, JSON)

Objective:

To Study:

1. **Loading of multiple file formats and performing data cleaning, transformation, and analysis.**
2. **Loading and analyzing sales data from CSV, Excel, and JSON formats.**

Theory:

Reading data and making it accessible is called data loading. Pandas provides various functions to read tabular data into a DataFrame.

Functions

Functions	Description
read_csv	Load data from a CSV file.
read_fwf	Read fixed-width column data.
read_clipboard	Read data from the clipboard.
read_excel	Read data from an Excel file.
read_html	Read tables from HTML documents.
read_json	Read data from a JSON file.
read_feather	Read Feather files.
read_orc	Read Apache ORC files.
read_parquet	Read Apache Parquet files.
read_pickle	Read Python pickle files.
read_sas	Read SAS datasets.
read_spss	Read SPSS data.

Functions	Description
read_sql	Read SQL query results.
read_sql_table	Read a SQL table.
read_stata	Read Stata datasets.
read_xml	Read XML files.

Indexing

Can use one or more columns as the DataFrame index and assign column names.

Type inference and data conversion

Supports automatic data type detection and conversion of values and missing-value markers.

Date and time parsing

Combines and converts date and time information into a single column.

Iterating

Supports processing large files in smaller chunks.

Unclean data issues

Data may contain missing values, extra rows, comments, or formatting issues. Pandas provides options to handle these problems during data loading.

Input

1. **Load sales data from CSV, Excel, and JSON files.**
2. **Explore the data structure and identify missing or inconsistent values.**
3. **Clean the data by handling missing values and duplicates.**
4. **Convert data into a common DataFrame format.**
5. **Perform transformations such as merging datasets and creating new variables.**
6. **Analyze sales using statistics, aggregations, and metrics.**
7. **Create visualizations such as bar plots, pie charts, and box plots.**

Applications:

1. **Real-time Analytics: Streaming data processing and instant insights.**
2. **Cloud Computing: Scalable storage and on-demand computing resources.**

Conclusion:

In this way we have successfully load and analyze sales data from different file formats, including CSV, Excel, and JSON, and perform data cleaning, transformation, and analysis on the dataset.

Experiment No. 2

Aim:

Data Cleaning and Preparation

Objective:

The goal is to perform data cleaning and preparation to gain insights into the factors that contribute to customer churn.

Theory:

Data cleaning and preparation is the process of preparing raw data for analysis. It includes identifying and removing errors, handling missing values, removing duplicates, correcting inconsistent data, and dealing with outliers. Proper data preparation ensures that the dataset is accurate, consistent, and ready for further analysis.

Steps involved in data cleaning and preparation:

Step 1: Remove duplicate or irrelevant observations.
Duplicate observations may occur when data is collected from multiple sources. Irrelevant records that are not useful for analysis should also be removed.

Step 2: Fix structural errors.
Structural errors occur due to inconsistent formats, spelling variations, or incorrect data types. These errors are corrected by applying consistent formatting rules.

Step 3: Filter unwanted outliers.
Outliers are data points that are significantly different from the rest of the dataset. They should be identified and handled properly as they can affect analysis results.

Step 4: Handle missing data.
Missing data can occur when information is not collected properly or is lost during processing. Missing values can be handled by deleting records, imputing values, or using suitable replacement methods.

Step 5: Validate and QA.
After cleaning, the dataset should be validated to check for remaining errors, inconsistencies, and outliers. Quality checks ensure that the data meets the required standards.

Input

1. **Use pandas.read_csv to load the dataset into a DataFrame.**
2. **Explore the dataset to understand its structure, columns, and contents.**
3. **Use isnull() and isna() to identify missing or null values.**

4. **Remove duplicate records from the dataset, if any.**
5. **Check for inconsistent data such as formatting and spelling variations and standardize it.**
6. **Identify and handle outliers by deleting, imputing, or transforming the affected values.**
7. **Perform feature engineering by creating new features relevant to predicting customer churn.**

Output:

Applications:

1. **Data Analysis: Cleaning, transforming, and modeling data to discover meaningful information.**
2. **Machine Learning: Preparing and cleaning data for developing accurate machine learning models.**
3. **Business Analytics: Cleaning, transforming, and consolidating data for business analysis and decision-making.**

Conclusion:

In this way, we explored the functions of the Python library for Data Cleaning and Preparation. We learned how to handle missing values, duplicate records, inconsistent data, and outliers in the Telecom Customer Churn dataset, making the data suitable for further analysis.

Experiment No. 3

Aim:

Data Wrangling on Real Estate Market

The dataset contains housing price information such as property characteristics, location, sale prices, and other relevant features. The goal is to perform data wrangling to understand factors influencing housing prices and prepare the dataset for analysis or modeling.

Dataset: "RealEstate_Prices.csv"

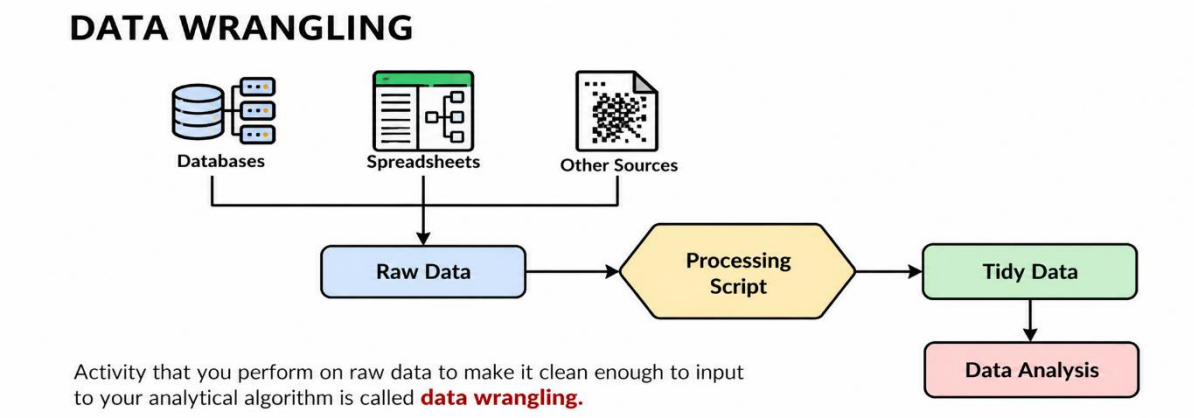
Objective:

Students should be able to perform data wrangling operations using Python on an open-source dataset.

Theory:

Data wrangling: Data wrangling, also known as data munging, is the process of cleaning, transforming, and organizing raw data for analysis and integration.

Data wrangling plays an important role in machine learning by preparing accurate, consistent, and error-free data.



1. Data Integration:

Combining data from multiple sources into a unified dataset by merging, cleaning, and transforming the data.

2. Data Visualization:

Creating visual representations such as scatter plots, histograms, and heat maps to understand patterns and relationships in the data.

3. Data Cleaning:

Identifying and correcting errors, inconsistencies, missing values, spelling errors, and duplicate records.

4. Data Reduction:

Reducing unnecessary or irrelevant data to decrease computational requirements and improve model performance.

5. Data Transformation:

Converting data into a suitable format by encoding categorical data, normalizing, or scaling values for analysis and modeling.

Input:

1. Import the "RealEstate_Prices.csv" dataset and clean the column names.
2. Handle missing values using suitable methods such as imputation or removal.
3. Merge additional datasets if relevant information is available.
4. Filter and subset data based on criteria such as time period, property type, or location.
5. Encode categorical variables using methods such as one-hot or label encoding.
6. Calculate summary statistics such as average sales prices by neighborhood or property type.
7. Identify and handle outliers or extreme values that may affect analysis.

Output:

Applications:

1. **Weather Data:** Data wrangling combines data from multiple sources and converts it into a suitable format for analysis.
2. **Social Media Data:** Wrangling techniques clean and transform unstructured or incomplete data for accurate analysis.

Conclusion:

In this way, we explored Python functions for **Data Preprocessing and Data Wrangling Techniques** and learned how to handle missing values in the **RealEstate_Prices Dataset**.

Experiment no 4

Aim :

Data Visualization using matplotlib

Problem Statement:

Analyzing Air Quality Index (AQI) Trends in a City Dataset:
"City_Air_Quality.csv"

Objective:

The dataset contains air quality measurements such as date, time, pollutant levels (PM2.5, PM10, CO), and AQI values. The goal is to use Matplotlib to visualize AQI trends and pollutant patterns in the city.

Theory:

Introduction: Data visualization is an important part of data analysis. It represents data graphically to identify trends, patterns, and insights. **Matplotlib** is a Python library used to create static and interactive visualizations.

1. Purpose of Data Visualization:

Data visualization is used for:

- **Summarization:** Converting large datasets into easy-to-understand visual representations.
- **Pattern Recognition:** Identifying trends, correlations, and outliers.
- **Decision Making:** Supporting informed decisions through visual data.
- **Communication:** Presenting complex data in a simple and effective way.

2. Matplotlib Overview:

Matplotlib is a flexible Python library used to create different types of plots, including:

- Line plots
- Bar charts
- Scatter plots
- Histograms
- Pie charts
- Heatmaps
- 3D plots

3. Basic Structure of a Matplotlib Plot:

A Matplotlib plot consists of:

- **Figure:** Overall window containing the plot.
- **Axes:** Area where data is plotted, including axes, labels, ticks, and gridlines.
- **Plot Elements:** Lines, bars, markers, or patches.
- **Legend:** Describes different data series.
- **Labels:** Define the content of the plot.

4. Pyplot Interface:

The Pyplot module provides simple functions to quickly create and manipulate plots.

5. Object-Oriented Interface:

The Object-Oriented interface provides greater control and flexibility for creating complex plots and customizing individual plot elements.

Applications of Data Visualization with Matplotlib:

- **Scientific Research:** Analyzing experimental data and patterns.
- **Business Analytics:** Presenting financial data, sales performance, and trends.
- **Machine Learning:** Visualizing model performance and data distributions.
- **Public Communication:** Presenting information through charts and graphs.

INPUT:

1. Import the "City_Air_Quality.csv" dataset.
2. Explore the dataset to understand its structure and contents.
3. Identify relevant variables such as date, pollutants, and AQI values.
4. Create line plots to visualize AQI trends over time.
5. Plot individual pollutant levels such as PM2.5, PM10, and CO.
6. Use bar or stacked bar plots to compare AQI values across dates.
7. Create box or violin plots to analyze AQI distributions.
8. Use scatter or bubble charts to study relationships between AQI and pollutant levels.
9. Customize plots using labels, titles, legends, and suitable colors.

OUTPUT:

CONCLUSION:

Matplotlib is a powerful and flexible Python library for data visualization. It allows users to create different types of plots and customize them easily. Using Matplotlib, raw data can be converted into meaningful visual insights, making analysis and communication easier.

Experiment No. 5

Aim

Data Aggregation

Objective

The main goal is to analyze and visualize retail sales data by aggregating and comparing sales across regions and product categories.

Dataset: "Retail_Sales_Data.csv"

Description

The dataset contains retail sales information such as transaction date, product category, quantity sold, and sales amount. The goal is to aggregate sales data to analyze performance by region and identify top-performing regions.

Theory

Data aggregation is the process of combining or summarizing data to produce a concise and meaningful representation. It helps identify trends, patterns, and relationships in the data.

Common aggregation operations include:

- **Sum:** Calculates the total values.
- **Mean:** Calculates the average values.
- **Median:** Finds the middle value.
- **Mode:** Finds the most frequently occurring value.
- **Count:** Counts the number of non-null values.

Examples of Data Aggregation:

- Identifying top-performing sales regions.
- Tracking website traffic by source.
- Summarizing data to gain useful insights and support decisions.

Benefits of Data Aggregation:

- **Improved efficiency:** Simplifies reporting and analysis.
- **Increased accuracy:** Reduces errors during manual processing.
- **Enhanced decision-making:** Provides a clearer view of business data.

Input

1. Import the "Retail_Sales_Data.csv" dataset using pandas.read_csv.
2. Explore the dataset structure and contents.
3. Identify relevant variables such as region, sales amount, and product category.
4. Group sales data by region and calculate total sales.
5. Create bar plots or pie charts to visualize sales distribution by region.

Output

Applications

1. Business Intelligence and Analytics
2. Healthcare
3. Financial Services
4. E-commerce and Retail

Conclusion

In this way, we implemented data aggregation to analyze retail sales performance by region and identify the top-performing regions. We also identified the top-selling product categories.

EXPERIMENT NO.6

Aim:

Analysis and Visualization of Stock Market Data Dataset:
"Stock_Prices.csv"

Objective:

The dataset contains historical stock price data including date, closing price, volume, and other relevant features. The goal is to analyze trends and patterns and develop models for forecasting future stock prices.

Theory:

Analysis and visualization of stock market data involve examining historical and real-time data to understand price movements, trends, and patterns. It helps identify market behavior and supports informed investment decisions.

1. Stock Market Data Analysis:

- **Technical Analysis:** Uses indicators such as moving averages, RSI, and Bollinger Bands to predict price movements.
- **Fundamental Analysis:** Examines a company's financial health to determine its intrinsic value.
- **Sentiment Analysis:** Analyzes news, social media, and investor behavior to understand market sentiment.

2. Data Visualization Techniques:

- **Line Charts:** Show stock price trends over time.
- **Candlestick Charts:** Display opening, high, low, and closing prices.
- **Bar Charts:** Visualize trading volume and market activity.
- **Moving Averages:** Smooth stock prices to identify trends.
- **Heatmaps:** Show the performance and correlation of multiple stocks.

3. Tools for Stock Market Analysis:

Popular tools include **Python, Matplotlib, Seaborn, Plotly, and Pandas**, along with APIs such as Alpha Vantage and Yahoo Finance for obtaining stock data.

Input:

1. Import the "Stock_Prices.csv" dataset.
2. Explore the dataset structure and contents.

3. Ensure the date column is in the correct format for time-series analysis.
4. Create line charts or time-series plots to visualize stock price trends.
5. Calculate and plot moving averages to identify trends and reduce noise.
6. Perform seasonal analysis to identify periodic patterns in stock prices.
7. Analyze the correlation between stock prices and variables such as trading volume.
8. Use ARIMA or exponential smoothing models to forecast future stock prices.

Output:

Conclusion:

Analyzing and visualizing stock market data helps identify market trends, monitor performance, and make data-driven financial decisions. These techniques are useful for forecasting, risk management, and portfolio analysis.